

Bringing smp into xpack-development

Step by step, snippet by snippet, in plain English

Official branch: xpack-development (3a22a06b)

Our branch: smp (1a728371)

How to bring smp into xpack-development, step by step

- 1. The situation
- 2. The rules
- 3. The official tests (the gate for every step)
- 4. What happens if we copy all the kernel code at once
- 5. How the code is organised, and why the order matters
- 6. How to make one step
- 7. The steps
- 8. The final merge, and what stays as xpack-development has it
- 9. One thing to know about GitHub
- 10. What can go wrong, and what to do
- 11. Summary in one sentence

How to bring `smp` into `xpack-development`, step by step

1. The situation

There are two versions of the same project, in `github.com/dan-maio/micro-os-plus-iii`:

branch	what it is	last commit
<code>xpack-development</code>	Liviu's official version. The kernel runs on one CPU core . Its tests must always pass.	<code>3a22a06b</code> (2026-09-20)
<code>smp</code>	Our version. It adds multi-core support (several CPU cores run threads at the same time), plus many bug fixes, new tests, new boards and documents.	<code>1a728371</code> (2026-09-29)

The kernel does not work alone. Every build uses a **port**, the part that knows the processor. The official tests use two ports:

port repository	used by	official version	our <code>smp</code> branch
<code>micro-os-plus-iii-posix-arch</code>	the 16 builds on the PC	v1.0.1	+18 commits
<code>micro-os-plus-iii-cortexm</code>	the 8 builds in QEMU	v1.1.0	+53 commits

Goal: move everything from `smp` (kernel **and** ports) into `xpack-development`, **without ever breaking the official tests**.

How far apart the kernel branches are:

- `smp` has **143 commits** that `xpack-development` does not have.
- `xpack-development` has **6 commits** that `smp` does not have. These are small changes by Liviu to `README-DEVELOPER.md` and `tests/package.json`.
- 317 files** are different. **43** of them are kernel code in `include/` and `src/`. All 43 are changes to existing files: `smp` adds no new kernel source file.

2. The rules

- A step is a few snippets of code, not whole files.** A snippet is one small piece: a few lines in one function, or one declaration.
- A step takes its snippets from the kernel and from the two ports together.** If the kernel needs something from the port, the same step brings it, in `posix-arch` and in `cortexm`.

3. **The steps follow the single-core / multi-core differences, one idea at a time:** first the bug fixes that also help one core, then the multi-core pieces one by one (per-CPU lock state, per-CPU current thread, kernel lock, ...).
4. **After every step, all existing tests of `xpack-development` must pass:** `xpm run test-all`, 72 tests, on the PC (posix-arch) and in QEMU (cortexm).
5. **The test procedure is never modified.** The existing test files, platforms and `package.json` actions stay as they are.
6. **New tests come at the end**, and they are only **added**: new files, new entries in CMake files, new configurations in `tests/package.json`.

3. The official tests (the gate for every step)

The official tests are in `tests/package.json` of `xpack-development`:

```
cd tests
npm install
xpm run install-all      # downloads compilers, QEMU, and the ports
xpm run test-all         # builds and runs every test
```

`test-all` builds the kernel in **24 ways**:

- **16 on the PC (Linux):** gcc 11, 12, 13, 14 and clang 16, 17, 18, 19, each in debug and release, with the port **posix-arch**.
- **8 in QEMU:** Cortex-M0, M3, M4F and M7, each in debug and release, with the port **cortexm**.

Each build runs **3 tests**: `rtos-apis`, `mutex-stress`, `cmsis-os-validator`. **A step is accepted only when all 72 pass.**

Important: the official build treats **every warning as an error** (`-Werror`), with many more warnings than our `smp` builds. Code that is clean on `smp` can fail here.

How the tests use our ports, without changing the tests

`install-all` downloads the **released** ports (posix-arch v1.0.1, cortexm v1.1.0). But a step also changes the ports. Upstream already has the official way to test local port sources: `xpm link`. It is described in `README-DEVELOPER.md` ("Use development writable packages"), and the actions are already in `tests/package.json`. No test file changes.

```
# once: register our two port folders, checked out on the step's branch
cd ~/Work/micro-os-plus/micro-os-plus-iii-posix-arch && git switch step/NN && xpm link
cd ~/Work/micro-os-plus/micro-os-plus-iii-cortexm && git switch step/NN && xpm link

# in the kernel's tests: use the linked ports instead of the downloaded ones
cd ~/Work/micro-os-plus/micro-os-plus-iii/tests
xpm run install-all

for c in native-cmake-gcc11 native-cmake-gcc12 native-cmake-gcc13 native-cmake-gcc14 \
    native-cmake-clang16 native-cmake-clang17 native-cmake-clang18 native-cmake-clang19 \
    qemu-cortex-m0-cmake-gcc qemu-cortex-m3-cmake-gcc \
    qemu-cortex-m4f-cmake-gcc qemu-cortex-m7f-cmake-gcc; do
    for t in debug release; do xpm run link-deps --config $c-$t; done
done
xpm run test-all # 72 / 72
```

Do not use `xpm run link-deps-all` **for this**. Its list in `tests/package.json` of `xpack-development` names `native-cmake-gcc13-debug` twice and **leaves out** `native-cmake-gcc14-debug`, so that build would keep testing the downloaded posix-arch v1.0.1. The loop above names exactly the 24 builds of `test-all`.

To be sure a build really uses the local port, look where its folder points after the loop:

```
ls -l tests/build/native-cmake-gcc14-debug/xpacks/@micro-os-plus/
# micro-os-plus-iii-posix-arch -> ~/Work/micro-os-plus/micro-os-plus-iii-posix-arch
```

4. What happens if we copy all the kernel code at once

On 2026-09-29 we tried it, in a temporary copy that has since been deleted, with the **released** ports:

1. `xpack-development` as it is, 8 of the 24 builds: **all 8 passed**.
2. With the kernel code from `smp` (`include/` and `src/`) on top: **all 8 failed to compile**. No test even ran.

There are exactly **5 reasons**:

#	where	what goes wrong	which step fixes it
1	<code>src/rtos/os-clocks.cpp</code> , <code>include/cmsis-plus/rtos/os-decls.h</code>	The kernel calls <code>port::clock_highres::has_hardware_counter()</code> . The released ports do not have it: "used but never defined".	Step 13 brings the kernel snippet and the port snippets together.
2	<code>src/libc/stdlib/timegm.c</code>	On <code>smp</code> the declaration of <code>timegm()</code> was removed on Linux, but the official Linux build needs it (<code>-Wmissing-prototypes</code>).	Step 1
3	<code>src/memory/first-fit-top.cpp</code> , <code>do_usable_size()</code>	A <code>char*</code> cast to a bigger type (32-bit ARM, <code>-Wcast-align</code>); pointer arithmetic (clang, <code>-Wunsafe-buffer-usage</code>).	Step 2
4	<code>src/libcpp/system-error.cpp</code>	Two <code>static</code> objects that live until exit (clang, <code>-Wexit-time-destructors</code>).	Step 3
5	<code>src/posix-io/file-descriptors-manager.cpp</code>	The new <code>descriptors_array__[files] == nullptr</code> tests (clang, <code>-Wunsafe-buffer-usage</code>).	Step 5

This is the main reason to go in small steps: each piece is polished for the stricter official build, one at a time.

5. How the code is organised, and why the order matters

In the kernel

Almost all the multi-core code is already inside `#if defined(OS_USE_SMP_SCHEDULER)`. The official builds do not define it, so the compiler **removes** that code. We checked it with the tool `unifdef`: removing every multi-core block from both branches leaves about **1 300 changed lines in 40 files**. These are the **bug fixes**. They change what one core does, so the existing tests really run them.

In cortexm

The files the QEMU builds compile are `include/cmsis-plus/rtos/port/*.h` and `src/rtos/os-core.cpp`. There, only **two** snippets change what one core does:

- `clock_highres::has_hardware_counter()` and `hardware_counter()` (returning `false` and `0`), which the kernel needs (reason 1);

- a real bug fix in `clock_highres::cycles_since_tick()`: it read the "tick pending" bit from `SysTick->CTRL` instead of `SCB->ICSR`.

Everything else in these files is inside `#if defined(OS_USE_SMP_SCHEDULER)`.

The Cortex-M33 and RP2350 code is in **new** files (`include-m33/`, `include-rp2350/`, `src/rtos/os-core-m33.cpp`, `src/rtos/os-core-rp2350.cpp`). The QEMU builds do not use them, so they come at the end with the new boards.

In posix-arch — the hard part

On `smp`, posix-arch was **rewritten** around one idea: *a host thread is a CPU*. Only **4** blocks are inside `#if defined(OS_USE_SMP_SCHEDULER)`. The rest is always on: `lock_state[OS_NCPU]`, `_in_isr[OS_NCPU]`, a `stack_ptr` field added to the thread context, the new file `src/host_cpu.cpp`, and a new `switch_stacks()`.

The 16 PC builds use posix-arch, so these changes **do** affect the existing tests. So in the steps below, every posix-arch multi-core snippet gets the guard that the `smp` code does not have:

```
#if defined(OS_USE_SMP_SCHEDULER)
    // the smp snippet
#else
    // the code exactly as in posix-arch v1.0.1
#endif
```

The single-core path stays the released code, which the 16 PC builds keep testing. The multi-core path grows step by step.

6. How to make one step

A step has **one branch with the same name** in each of the three repositories, all made from their `xpack-development`:

```
for r in micro-os-plus-iii micro-os-plus-iii-posix-arch micro-os-plus-iii-cortexm; do
    git -C ~/Work/micro-os-plus/$r fetch origin
    git -C ~/Work/micro-os-plus/$r switch -c step/04-lock-state origin/xpack-development
done
```

Take **only the snippets of this step**, not the whole file:

```
cd ~/Work/micro-os-plus/micro-os-plus-iii-cortexm
git diff origin/xbpack-development smp -- include/cmsis-plus/rtos/port/os-decls.h > /tmp/s.patch
# edit /tmp/s.patch: keep only the lock_state hunk
git apply --index /tmp/s.patch
git commit -m "feat(smp): per-CPU scheduler lock state"
```

Then run the gate of section 3 (`xpm link` both ports, `link-deps-all`, `test-all`), which must show **72 / 72**.

Then open **one PR per repository**, each saying in its description which PRs in the other repositories belong to the same step. **They are merged together**: the port PRs first, then the kernel PR.

After each step is merged, the `smp` branches are updated with the new `xbpack-development` (`git merge origin/xbpack-development`), so the difference gets smaller after every step.

7. The steps

In the tables, **K** is the kernel (`micro-os-plus-iii`), **C** is `cortexm`, **P** is `posix-arch`. "—" means that repository has nothing in this step.

Part A — bug fixes that also help one core (steps 1 to 13)

The existing tests run this code, so these steps are really tested.

step	idea	K (kernel snippets)	C	P
1	build fixes	<code>posix/dirent.h</code> : empty <code>struct DIR</code> gets a member. <code>timegm.c</code> : declare <code>timegm()</code> unless glibc declares it (reason 2, see below). <code>c-syscalls-aliases-standard.h</code> : <code>read()</code> / <code>write()</code> return type from newlib. <code>utils/lists.h</code> : iterator calls <code>next()</code> / <code>prev()</code> . <code>os-thread.cpp</code> : <code>this_thread::suspend()</code> not <code>inline</code> . <code>os-decls.h</code> : <code>#include <cmsis-plus/rtos/os-c-decls.h></code> .	—	—
2	memory	<code>os-memory.h</code> : <code>align_size()</code> overflow; new <code>usable_size()</code> / <code>do_usable_size()</code> . <code>os-memory.cpp</code> : default <code>do_usable_size()</code> returns 0. <code>first-fit-top.cpp</code> : size checks in <code>do_allocate()</code> ; <code>do_usable_size()</code> with the warnings silenced (reason 3). <code>lifo.cpp</code> : size checks; a too-small head chunk. <code>block-pool.cpp</code> : the inverted <code>if</code> . <code>malloc.cpp</code> : <code>calloc()</code> overflow; <code>realloc()</code> copies <code>min(old, new)</code>.	—	—
3	C++ library	<code>new.cpp</code> : the 10 <code>align_val_t</code> operators (4 <code>new</code> , 6 <code>delete</code>). <code>system-error.cpp</code> : static categories, warning silenced (reason 4).	—	—

step	idea	K (kernel snippets)	C	P
		<pre> estd/ memory_resource : select_on_containe r_copy_constructio n() returns *this . chrono.cpp : seconds + remainder. </pre>		
4	C / CMSIS API	<pre> os-c-wrapper.cpp : os_timer_create() / os_timer_new() default to one-shot; os_mutex_delete() / os_semaphore_delet e() delete the concrete type; (uint64_t) millisec * 1000u (10 places). </pre>	—	—
5	files	<pre> file-system.h , net-stack.h : deferred lists under interrupts::critic al_section . file- descriptors- manager.cpp : every function under the lock, null checks, warning silenced (reason 5). block- device.cpp : the three != 0 → == 0 . </pre>	—	—
6	semihosting, ARMv8-M	<pre> arm/ semihosting.h : ARMv8-M uses bkpt ; SEMIHOST_TRAP_HLT . c-syscalls- semihosting.cpp : fstat() keeps the type; weak os_board_console_m irror() . cortexm/ exception- handlers.h , startup/exception- handlers.c : __ARM_ARCH_8M_MAIN — next to 7M/7EM. </pre>	—	—
7	timers	<pre> os-lists.cpp check_timestamp() : unlink under the lock, </pre>	—	—

step	idea	K (kernel snippets)	C	P
		call <code>action()</code> after. <code>os-timer.cpp</code> <code>internal_interrupt</code> <code>_service_routine(</code> <code>)</code> : no burst; link under the lock. <code>os-</code> <code>thread.h</code> <code>__errno()</code> : a scratch <code>int</code> in handler mode.		
8	mutex	<code>os-mutex.cpp</code> : <code>internal_try_lock_</code> <code>()</code> ceiling check before owning; inherit keeps the maximum; the owner captured before the uncritical section. <code>internal_unlock_(</code> <code>)</code> : recompute the inherited priority.	—	—
9	thread life cycle	<code>os-c-decls.h</code> , <code>os-thread.h</code> : state <code>destroying = 7</code> . <code>os-thread.cpp</code> : <code>resume()</code> only from suspended/initializing; the two priority relinks under one lock; <code>detach()</code> ; <code>join()</code> under one lock; <code>internal_destroy_(</code> <code>)</code> wakes the joiner under the lock; <code>kill()</code> claims the thread once. <code>os-</code> <code>idle.cpp</code> : skip <code>destroying</code> / <code>destroyed</code> . <code>os-c-</code> <code>wrapper.cpp</code> : its <code>static_assert</code> .	—	—
10	condition variable	<code>os-condvar.h</code> , <code>os-c-decls.h</code> : the <code>clock</code> member. <code>os-condvar.cpp</code> : <code>wait()</code> / <code>timed_wait()</code> link and unlock in one step. <code>instrumentation.h</code> : <code>SUSPEND_CAUSE_COND</code> <code>VAR</code> . <code>estd/</code> <code>condition_variabl</code>	—	—

step	idea	K (kernel snippets)	C	P
		<code>e : wait_for()</code> trusts the kernel's result.		
11	<code>std::thread</code>	<code>estd/</code> <code>thread_internal.h</code> : remember <code>function_object_</code> . <code>thread-cpp.h</code> : <code>join()</code> waits, then frees.	—	—
12	message queue	<code>os-mqueue.cpp</code> : <code>port::scheduler::reschedule()</code> after a send/receive that woke a waiter (6 places).	—	—
13	hardware counter	<code>os-decls.h</code> : declare <code>has_hardware_counter()</code> and <code>hardware_counter()</code> . <code>os-clocks.cpp</code> <code>clock_highres::now()</code> : use it when the port has one (reason 1).	<code>os-inlines.h</code> : both functions, returning <code>false / 0</code> . Plus the <code>cycles_since_tick()</code> fix: <code>SCB->ICSR</code> , not <code>SysTick->CTRL</code> .	<code>os-inlines.h</code> / <code>os-core.cpp</code> : both functions, reading <code>CLOCK_MONOTONIC</code> (smp commit <code>53eacf4</code> , "monotonic hrclock").

Step 13 no longer needs an `#if` in the kernel: the ports bring the two functions **in the same step**.

The snippet for step 1, `timegm.c` , which is **different from** `smp` :

```
// newlib has no timegm(). glibc's <time.h> declares it only under
// `__USE_MISC || __GLIBC_USE (ISOC23)`; the official PC builds use
// _POSIX_C_SOURCE / _XOPEN_SOURCE and C11, so there it is NOT declared.
// Apple's libc always declares it. __GLIBC_USE is a glibc macro, so it is
// tested only inside the glibc branch.
#if defined(__GLIBC__)
#if !(defined(__USE_MISC) || __GLIBC_USE (ISOC23))
time_t
timegm (struct tm* tim_p);
#endif
#elif !defined(__APPLE__)
time_t
timegm (struct tm* tim_p);
#endif
```

The warning snippets for steps 2, 3 and 5 use the pattern the files already use elsewhere:

```

#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wcast-align"           // step 2 only
#if defined(__clang__)
#pragma clang diagnostic ignored "-Wunsafe-buffer-usage" // steps 2 and 5
// #pragma clang diagnostic ignored "-Wexit-time-destructors" // step 3
#endif
    // ... the smp code, unchanged ...
#pragma GCC diagnostic pop

```

Every one of these snippets is also put on `smp`, so the branches stay the same.

Order: step 1 first. Step 9 after step 7. Steps 10 and 11 after step 9. The others can go in any order.

Part B — the multi-core pieces, one idea at a time (steps 14 to 23)

Every snippet here is inside `#if defined(OS_USE_SMP_SCHEDULER)` (for posix-arch, with the `#else` keeping the v1.0.1 code, see section 5). The official builds do not define it, so the 72 tests must stay green, and they prove that **nothing changed for one core**.

That also means the official tests **do not compile** these snippets. The multi-core pieces only work together: for example, cortexm's `switch_stacks()` multi-core branch needs the kernel lock (step 16), the per-CPU current thread (step 18) and the saved-context rule (step 21) at the same time. So a multi-core build is not expected to compile in the middle of Part B. Instead there is **one extra check at the end of Part B** (after step 23), outside the gate: build and run our own `smp` multi-core tests (the native multi-core build and `2xcortex-m33`) against the three step branches. If it fails, the missing snippet is found and added to its step before the PRs are merged.

The single-core proof for every Part B step:

```

for f in $(git diff --name-only origin/xpack-development HEAD); do
    git show origin/xpack-development:$f | unifdef -UOS_USE_SMP_SCHEDULER > /tmp/old
    git show HEAD:$f | unifdef -UOS_USE_SMP_SCHEDULER > /tmp/new
    diff /tmp/old /tmp/new      # must print nothing
done

```

step	idea	K (kernel snippets)	C (cortexm snippets)	P (posix-arch snippets)
14	how many CPUs, and which one am I	<code>os-core.cpp</code> and <code>os-thread.cpp</code> : <code>extern "C"</code> <code>unsigned</code> <code>port_cpu_id</code> <code>(void);</code>	<code>os-inlines.h</code> : <code>port_cpu_id()</code> returns 0. <code>os-core.cpp</code> : the <code>extern "C"</code> <code>port_cpu_id()</code> wrapper; <code>static_assert</code> <code>(OS_NCPU == 1)</code> .	<code>os-decls.h</code> : <code>OS_NCPU</code> defaults to 1. <code>os-core.cpp</code> : <code>thread_local</code> <code>_this_cpu</code> ; <code>extern</code> <code>"C"</code> <code>port_cpu_id()</code> , <code>noinline</code> , with the compiler barrier (the clang 16–18 fix, smp commit <code>3159a57</code>).
15	one scheduler lock state per CPU	—	<code>os-decls.h</code> : <code>lock_state[OS_NCPU]</code> <code>]</code> , <code>lock_primask[OS_NCPU]</code> <code>PU]</code> . <code>os-inlines.h</code> : <code>locked()</code> reads <code>lock_state[port_cpu_id()]</code> . <code>os-core.cpp</code> : the array; <code>scheduler::locked</code> <code>(state_t)</code> and <code>start()</code> index it.	<code>os-decls.h</code> : <code>volatile</code> <code>lock_state[OS_NCPU]</code> <code>]</code> . <code>os-core.cpp</code> : <code>locked (state_t)</code> blocks the tick before reading the CPU id (smp commit <code>191f89d</code>).
16	the kernel lock	—	<code>os-c-decls.h</code> : <code>SMP_NO_OWNER</code> . <code>os-decls.h</code> : <code>struct</code> <code>smp_klock_t</code> , <code>_smp_klock</code> . <code>os-inlines.h</code> : <code>_smp_klock_enter</code> <code>() /</code> <code>_smp_klock_exit()</code> ; called from <code>critical_section::</code> <code>enter()</code> / <code>exit()</code> . <code>os-core.cpp</code> : the definition.	<code>os-decls.h</code> : <code>SMP_NO_OWNER</code> , the same struct and names. <code>os-core.cpp</code> / <code>os-inlines.h</code> : the recursive lock and its enter/exit in the critical section.
17	interrupts are per CPU	—	— (one CPU)	<code>os-decls.h</code> : <code>#if</code> multi-core <code>irq_set</code> , <code>_in_isr[OS_NCPU]</code> , <code>signal_nesting</code> ; <code>#else</code> the v1.0.1 <code>clock_set</code> (on smp , <code>irq_set</code> replaced <code>clock_set</code>). <code>os-inlines.h</code> : <code>in_handler_mode()</code> reads <code>_in_isr[port_cpu_i</code>

step	idea	K (kernel snippets)	C (cortexm snippets)	P (posix-arch snippets)
				<code>d())</code> or <code>signal_nesting</code> (smp commit <code>cd1a728</code>); the critical section blocks <code>irq_set</code> .
18	one current thread per CPU	<code>os-sched.h</code> , <code>os-core.cpp</code> : <code>current_thread_[OS_NCPU]</code> ; the two statistics lines use <code>[port_cpu_id()]</code> . <code>os-thread.cpp</code> : <code>current_thread_[port_cpu_id()] = this</code> . <code>os-thread.h</code> : <code>this_thread::thread()</code> reads it with interrupts masked (smp commit <code>3008ed64</code>).	<code>os-core.cpp</code> <code>start()</code> : <code>current_thread_[0] = pth</code> .	<code>os-core.cpp</code> <code>start()</code> : <code>current_thread_[cpu]</code> .
19	one idle thread per CPU	<code>os-core.cpp</code> : <code>os_idle_thread_core[OS_NCPU]</code> .	<code>os-core.cpp</code> <code>start()</code> : <code>os_idle_thread_core[0] = ::os_idle_thread</code> .	<code>os-core.cpp</code> <code>start()</code> : the same for CPU 0.
20	which CPUs a thread may run on	<code>os-c-decls.h</code> : <code>th_cpu_affinity</code> , <code>cpu_affinity</code> . <code>os-thread.h</code> : the attribute, <code>cpu_affinity()</code> get/set, the member (default <code>0xFFFFFFFF</code>). <code>os-thread.cpp</code> : the three initialisations and the get/set. <code>os-core.cpp</code> : <code>is_thread_allowed_on_cpu()</code> . <code>os-main.cpp</code> : main pinned to CPU 0. <code>os-c-wrapper.cpp</code> : CMSIS-RTOS v1 threads pinned to CPU 0 (2 places).	—	—
21		<code>os-thread.h</code> : <code>os_rtos_idle_actio</code>	<code>os-core.cpp</code> <code>switch_stacks()</code> :	<code>os-c-decls.h</code> : <code>stack_ptr</code> first in

step	idea	K (kernel snippets)	C (cortexm snippets)	P (posix-arch snippets)
	"this context is saved, another CPU may take it"	<code>ns()</code> becomes a <code>friend</code> . <code>os-idle.cpp</code> : the reaper skips a thread that is still live (<code>stack_ptr == nullptr</code> or current on a CPU). <code>os-thread.cpp</code> : <code>join()</code> and <code>kill()</code> wait until the thread is off every CPU.	its whole multi-core branch (kernel lock, <code>current_thread_[cpu]</code> , store the old SP, clear the new one's <code>stack_ptr</code>).	<code>os_port_thread_context_t</code> . <code>os-core.cpp</code> <code>switch_stacks()</code> : the deferred publish.
22	the multi-core picker	<code>os-thread.h</code> : <code>internal_switch_threadreads()</code> becomes a <code>friend</code> . <code>os-core.cpp</code> <code>internal_switch_threadreads()</code> : the affinity-aware pick from the ready list, falling back to the CPU's idle thread.	—	—
23	more than one CPU on the PC	<code>os-thread.cpp</code> : the weak, empty <code>port_smp_ipi()</code> .	— (one CPU on the generic Cortex-M)	<code>src/host_cpu.cpp</code> , <code>include/host_cpu.hpp</code> : a host thread per CPU, its tick timer, the IPI signal, starting the other CPUs, and the strong <code>port_smp_ipi()</code> . These are new files ; their whole content is inside <code>#if defined(OS_USE_SMP_SCHEDULER)</code> , and one line adds <code>src/host_cpu.cpp</code> to the <code>target_sources</code> list in the port's <code>CMakeLists.txt</code> .

After step 23, `include/` and `src/` of the kernel are **the same** on both branches, and so are the generic `cortexm` files and `posix-arch`, apart from the `#else` single-core code, which `smp` gets back in its own update.

Part C — at the end: new cores, new tests, new boards (steps 24 to 28)

Only now are new tests added, and only **added**: the existing tests, platforms and actions do not change.
The gate is: **the 72 old tests still pass**, and the new tests of the step pass too.

step	what	how it stays add-only
24	release the ports	Tag <code>posix-arch</code> v1.1.0 and <code>cortexm</code> v1.2.0. Create <code>micro-os-plus-iii-devices</code> upstream (it does not exist in <code>micro-os-plus</code> yet) and tag v1.0.0. Bring <code>aarch32</code> and <code>aarch64</code> in the same way (Livi has 6 and 7 newer commits there; take them first). The old configurations keep asking for <code>posix-arch</code> v1.0.1 and <code>cortexm</code> v1.1.0.
25	new cores and board support	<code>cortexm</code> : <code>include-m33/</code> , <code>include-rp2350/</code> , <code>src/rtos/os-core-m33.cpp</code> , <code>src/rtos/os-core-rp2350.cpp</code> , <code>src/libc/getentropy.c</code> , <code>src/semihosting-exit.cpp</code> . <code>posix-arch</code> : <code>board-contract.cpp</code> , <code>exception_handler.{hpp,cpp}</code> , <code>free-store.cpp</code> , <code>hw_result.hpp</code> (used only by the new test harness). New files, and new targets in the ports' <code>CMakeLists.txt</code> ; the existing targets are not changed. Kernel: the wake-up IPI in <code>thread::resume()</code> (the <code>OS_INTEGER_RTOS_PORT_NCPU > 1</code> block). Only the RP2350 defines <code>OS_INTEGER_RTOS_PORT_NCPU</code> , so this snippet comes with it.
26	build files	Kernel: <code>cmake/toolchains/</code> , <code>cmake/uos-app.cmake</code> , <code>port/smp-common/</code> , <code>tools/</code> : new files. The smaller kernel targets (<code>micro-os-plus::iii-posix-io</code> , <code>micro-os-plus::iii-drivers</code> , ...) as new names; <code>micro-os-plus::iii</code> keeps meaning "all kernel files".
27	new test sources and runners	<code>tests/sources/fp-switch/</code> , <code>test_smpl/</code> , <code>tests/device-qemu-cortexm/linker-scripts/mem-mps2-an505.ld</code> and <code>mem-mps2-an521.ld</code> : new files.
28	new test platforms	New folders <code>tests/platforms/<name>/</code> , one platform per commit: <code>pico2-1cpu</code> , <code>2xcortex-m33</code> , <code>aarch32-rpi3b</code> , <code>aarch32-rpi-zero-2w</code> , <code>aarch64-rpi3b</code> , <code>aarch64-rpi-zero-2w</code> , <code>cortexm-pico2</code> , a native multi-core platform; and the hardware ones (marked <code>hwd</code> , never run by <code>xpm run test</code>). <code>tests/cmake/tests-main.cmake</code> : add one <code>elseif</code> per new name; the last

step	what	how it stays add-only
		<code>else</code> stays the old code, so <code>native</code> and <code>qemu-cortex-*</code> build exactly as before. <code>tests/package.json</code> : add the new configurations and new actions (for example <code>test-smp-all</code>); <code>test-all</code> does not change.

Then:

- **Step 29, documents:** `docs/tests/`, `docs/STATUS.md`, the port documents, and their PDFs (new files). Ask Liviu whether he wants the review reports (`docs/DeepSeek-review.*`, `docs/agy-review.*`).
- **Step 30, the final merge.** See section 8.

8. The final merge, and what stays as `xpack-development` has it

Before the final merge, `smp` itself is made to agree with the rules:

1. Take Liviu's 6 newer commits: `git merge origin/xpack-development`.
2. Take back Liviu's version of everything in the tables below.
3. Stop tracking the local links in `tests/xpacks/`.
4. Run the `smp` tests (the new platforms) once more.

Then `git diff --stat origin/xpack-development smp` must show **nothing** (or only things we chose to keep on `smp`). If anything else shows up, it was forgotten: make a small step for it first. Then:

```
gh pr create --repo dan-maio/micro-os-plus-iii --base xpack-development \
  --head smp --title "Integrate smp"
```

and run the 72 old tests and all the new tests one last time.

Files `smp` deleted or renamed, which Liviu needs:

on <code>smp</code>	why it stays
<code>.github/workflows/ci.yml</code> is deleted	Liviu's automatic test on GitHub.
<code>README.md</code> is deleted	The front page of the project.
<code>LICENSE</code> is renamed to <code>LICENSE-micro-os-plus-iii</code>	GitHub looks for <code>LICENSE</code> .
<code>doxygen/</code> , <code>inspiration/</code> , <code>templates/</code> are deleted	Liviu's reference manual and material.
<code>docs/HISTORY.md</code> , <code>docs/NOTES.md</code> , <code>docs/TODO.md</code> and 3 more are deleted	Liviu's documents.
<code>.vscode/</code> , <code>.settings/</code>	Editor settings.

Existing test files `smp` changed (the rules say: do not modify the test procedure):

on <code>smp</code>	what <code>smp</code> changed
<code>tests/platforms/native/</code> , <code>tests/platforms/qemu-cortex-m{0,3,4f,7f}/</code>	rewritten to build through the new ports
<code>tests/platforms/{nucleo-*,raspberrypi-pico}/</code>	small additions
<code>tests/CMakeLists.txt</code>	the project name includes the platform
<code>tests/cmake/tests-main.cmake</code>	<code>native</code> and <code>qemu-cortex-*</code> go through the new ports (in step 28 only new names do)
<code>tests/sources/rtos-apis/</code> , <code>tests/sources/cmsis-os-validator/</code>	bigger heap, no libucontext, a switch to skip FatFs
<code>tests/device-qemu-cortexm/</code> (existing files)	FLASH size of an385/an386, M33 vectors
<code>tests/package.json</code> (existing entries)	changed actions and configurations
<code>tests/xpacks/*</code>	links to <code>/home/dan/.local/xPacks/...</code> , only valid on our PC

If a new platform needs one of these changes, it gets it through **its own new files** (for example its own `os-app-config.h` in its own folder).

Decision for you: the an385/an386 FLASH size (8 MB → 128 MB) is a real bug fix, but it is in an existing test file. Either send it as a separate, clearly labelled PR that Liviu can accept or refuse, or leave it out.

9. One thing to know about GitHub

Our repositories (`dan-maio/...`) are **not forks** of Liviu's (`micro-os-plus/...`) on GitHub.

- We **can** open PRs inside our own repositories (into their `xpack-development` branches) right now.
- To send them to **Liviu**, we must first create real forks of his repositories, and push the step branches there.

10. What can go wrong, and what to do

problem	what to do
A step fails a test because it contains a snippet of another step.	Compare with <code>git diff</code> (and <code>unifdef</code> in Part B) and move the snippet.
A kernel fix needs a test change to pass.	Not allowed. The fix is wrong for one core, or it belongs behind <code>#if defined(OS_USE_SMP_SCHEDULER)</code> ; change the fix, not the test.
The kernel PR of a step is merged but a port PR is not.	Never merge them apart: ports first, then the kernel, in the same session.
A build still uses the downloaded port (section 3).	Use the explicit <code>link-deps</code> loop, not <code>link-deps-all</code> (which skips <code>native-cmake-gcc14-debug</code>), and check the <code>xpacks/@micro-os-plus/</code> link.
A Part B snippet is missing, so the multi-core build fails at the end of Part B.	The official tests cannot see it; the extra multi-core check after step 23 finds it. Add it to its step before the PRs are merged.
Liviu changes <code>xpack-development</code> while steps are open.	Update each open branch with his version (<code>git merge origin/xpack-development</code>).
A QEMU test takes too long on a slow computer (timeout).	Run that one test alone before calling it a failure. Never run several QEMU tests at the same time.

11. Summary in one sentence

Bring in small snippets of kernel and port code together — first the bug fixes that help one core, then the multi-core pieces one idea at a time, each hidden behind `OS_USE_SMP_SCHEDULER` — checking the 72 existing tests after every step without changing them; at the end, release the ports and add the new cores, tests and boards as new files only.